

-

multysevice ועוד. וגם על מערכות task thread שאלו על תהליכים מקבילים לעומק

Senior / Team Lead #C/.NET – זה כבר אזור מאוד קלאסי של ריאיון ל-

בדרך **Multi-Service** וגם **מערכות Thread, Task**, כששואלים אותך **על תהליכים מקבילים** כלל לא בודקים רק הגדרות, אלא האם אתה באמת מבין:

1. מה רץ איפה
2. איך מונעים בעיות
3. איך מערכת מתנהגת בעומס
4. איך חושבים ארכיטקטונית

אני אסביר לך את זה כמו לריאיון.

מה ההבדל — Process, Thread, Task

Process

הוא מופע רץ של תוכנה **Process**.

למשל:

- Process = אחת Web API אפליקציית
- Process = נוסף Worker Service אחר
- Process = אחר SQL Server

יש Process לכל:

- זיכרון נפרד
- משאבים משלו
- אחרים Processes בידוד מ־

במילים פשוטות:

תוכנית שלמה שרצה בפני עצמה = Process

Thread

Thread הוא יחידת ביצוע בתוך Process.

Threads יכולים להיות כמה Process בתוך אותו:
הם חולקים:

- Process אותו זיכרון של ה-
- אותם אובייקטים
- resources אותם

מבצע קוד משלו Thread אבל כל

במילים פשוטות:

Thread = נתיב ביצוע בתוך תוכנה

דוגמה:

- אחד מטפל בבקשה Thread
- אחר עושה עבודה ברקע Thread
- O/Aחר מחכה ל- Thread

Task

לעבודה אסינכרונית או מקבילית NET. הוא אבסטרקציה גבוהה יותר ב- Task.

Thread הוא לא בהכרח Task.

חשוב מאוד לריאיון:

Task ≠ Thread

אומר Task:

"יש לי יחידת עבודה שאני רוצה להריץ"

מחליט runtime:

- Thread Pool האם להשתמש ב־
- בזמן ההמתנה האם זו עבודה אסינכרונית בלי להחזיק
- מתי לתזמן אותה

במילים פשוטות:

- **Thread** = מי באמת מבצע
- **Task** = העבודה הלוגית שאתה מבקש להריץ

משפט חשוב לריאיון 2)

אם ישאלו אותך:

Thread ל־Task מה ההבדל בין

תענה משהו כזה:

הוא משאב מערכת הפעלה שמבצע קוד בפועל Thread.

NET. של עבודה אסינכרונית או מקבילית ב־ abstraction הוא Task.

ובמקרה של Thread Pool חדש — בדרך כלל הוא משתמש ב־ Thread לא בהכרח יוצר Task בזמן ההמתנה Thread אסינכרוניות הוא יכול אפילו לא להחזיק I/O פעולות.

זו תשובה מאוד חזקה.

Task ומתי ב־ Thread מתי משתמשים ב־ (3)

בדרך כלל

מוזרני כמעט תמיד תעדיף ב־ .NET:

- `Task`
- `async/await`
- `Task.Run` רק כשבאמת צריך

ידנית Threads ולא תנהל.

פחות מומלץ Thread מתי

Thread יצירה ידנית של:

- יקרה יותר
- קשה יותר לניהול
- `scalable` פחות
- פחות נפוצה בקוד אפליקטיבי רגיל

Thread מתי כן אולי תשתמש ב־

רק במקרים מיוחדים:

- Thread שליטה מלאה על
- Thread long-running מאוד מיוחד
- low-level סטייל ישן או
- dedicated thread תרחישים נדירים של

ברוב הראיונות, אם תגיד

ידני Thread על פני `async/await` Task היום בדרך כלל אני מעדיף

זה נשמע נכון ובוגר.

4) CPU-bound מול I/O-bound

זה אחד הדברים הכי חשובים

CPU-bound

עבודה שצורכת מעבד:

- חישובים כבדים
- עיבוד תמונה
- עיבוד אלגוריתמי
- הצפנה
- כבד parsing

במקרה כזה:

- יש משמעות להרצה מקבילית
- אפשר להשתמש ב- `Parallel.For`
- אפשר להשתמש ב- `Task.Run`

I/O-bound

עבודה שמחכה ל

- DB
- HTTP
- קבצים
- Redis
- Message Queue

במקרה כזה:

- שסתם יעמוד ויחכה Thread לא צריך
- `async/await` עדיף

משפט טוב לריאיון:

background workers ל-offloading או parallelism אני שוקל CPU-bound ל-
threads כדי לא לחסום `async/await` אני מעדיף I/O-bound ל-.

מה זה Thread Pool 5)

שמנוהלים אוטומטית Threads מאגר — **Thread Pool** יש ב-.NET

חדש Thread במקום ליצור כל פעם:

- Threads המערכת ממחזרת
- זה יותר יעיל
- זה יותר מהיר
- scalable זה יותר

Thread Pool משתמשים ב-.NET ב-Tasks רוב ה-

זה חשוב כי בריאיון לפעמים שואלים:

חדש Thread פותח Task.Run?

התשובה הנכונה:

Thread Pool חדש. בדרך כלל העבודה מתוזמנת ל- Thread לא בהכרח.

6) מה זה Race Condition

או יותר ניגשים לאותו מידע במקביל Threads כששני, והתוצאה תלויה בסדר הריצה שלהם.

דוגמה קלאסית:

```
int x = 0;

Parallel.For(0, 1000, i =>
{
    x++;
});
```

לכאורה מצפים ל-1000, אבל בפועל תקבל ערך לא צפוי.

למה?

זו לא פעולה אטומית `x++` כי

היא למעשה:

1. קריאה של x
2. חישוב x+1
3. כתיבה חזרה

"עושים את זה יחד, חלק מהעדכונים "נאכלים" threads אם שני.

7) Race Condition איך פותרים

lock

```
object obj = new object();
int x = 0;
```



```
Parallel.For(0, 1000, i =>
{
    lock (obj)
    {
        x++;
    }
});
```

Interlocked

```
int x = 0;

Parallel.For(0, 1000, i =>
{
    Interlocked.Increment(ref x);
});
```

בריאיון חשוב לדעת:

- `lock` = מונע כניסה בו־זמנית לקטע קריטי
- `Interlocked` = מתאים לפעולות אטומיות פשוטות ומהיר יותר

8) מה זה Deadlock

Deadlock (או יותר) מחכים אחד לשני לנצח threads הוא מצב שבו שני

דוגמה רעיונית:

- Thread A נועל resource 1 ומחכה ל־ resource 2
- Thread B נועל resource 2 ומחכה ל־ resource 1

אף אחד לא מתקדם.

בריאיון אוהבים לשאול:

deadlock איך מונעים?

תשובה טובה:

- סדר נעילות קבוע
 - להקטין קטעים נעולים
 - להימנע מנעילות מיותרות
 - כשלא צריך async על `Result` ו-`Wait()` להימנע מ-
 - כשאפשר timeouts להשתמש ב-
-

9) Thread-safe מה זה

בלי לשבור threads הוא כזה שאפשר לגשת אליו ממספר **Thread-safe** קוד או אובייקט שהוא את המידע.

דוגמאות:

- `ConcurrentDictionary`
- `Interlocked`
- `lock`

thread-safe אם מחלקה לא:

- race conditions ייתכנו
 - corruption
 - תוצאות אקראיות
-

10) Concurrent collections

רגיל במקביליות, עדיף לעיתים `Dictionary` במקום

- `ConcurrentDictionary`
- `ConcurrentQueue`
- `ConcurrentBag`

למה?

`threads` כי הן בנויות לעבודה בטוחה בסביבה מרובת

אבל בריאיון חשוב גם להגיד:

`concurrency` לא פותרות כל בעיית `concurrent collections`.
הן עוזרות במבני נתונים, אבל עדיין צריך לחשוב על הלוגיקה הכוללת.

זו תשובה בוגרת.

מה באמת קורה — `async/await` 11)

`async/await` הרבה מועמדים יודעים לכתוב,
אבל לא יודעים להסביר.

הסבר פשוט לריאיון:

מאפשר לכתוב קוד אסינכרוני בצורה קריאה `async/await`.
יכול להשתחרר ולעשות עבודה אחרת `thread` - O/A, וכאשר פעולה אסינכרונית מחכה ל-
כאשר הפעולה מסתיימת, המשך המתודה ממשיך.

דגש חשוב:

- `await` "חדש `thread` לא אומר "פתח"
- `await` "לא אומר "מקבילי"
- "הוא אומר "אל תחסום עד שהתוצאה תחזור"

מה ההבדל בין מקביליות ל-אסינכרוניות (12)

זה אחד השאלות האהובות.

Concurrency

יכולת לטפל בכמה משימות "יחד" מבחינה ניהולית.

Parallelism

כמה משימות באמת רצות פיזית באותו זמן.

Asynchrony

לא לחכות בצורה חוסמת לפעולה חיצונית.

ניסוח יפה לריאיון:

O/I ובמיוחד ב- threads, אסינכרוניות עוזרת לנצל טוב יותר

מקביליות עוזרת לבצע כמה עבודות במקביל.

הוא מקרה שבו כמה עבודות באמת רצות בו זמנית על כמה ליבות Parallelism.

Task / Thread שאלות ריאיון נפוצות על (13)

נפרד Thread רץ על Task שאלה: האם כל

לא.

חדש Thread יוצר async שאלה: האם

לא בהכרח.

ASP.NET תמיד נכון ב- Task.Run שאלה: האם

לא תמיד.

למה?

אמיתי async עדיף פשוט, I/O אם זו פעולת

בלי צורך Thread Pool יכול סתם לזרוק עבודה ל- `Task.Run`.

Task.Run שאלה: מתי? כן יכול להיות שימושי

- CPU-bound עבודה
- של עבודה כבדה offload
- קוד חוסם ישן שאתה לא יכול לשנות מיד

14) CancellationToken מה זה

במערכות רציניות שואלים על זה הרבה.

זה מנגנון שמאפשר לבטל עבודה בצורה מסודרת

למשל:

- בוטלה HTTP בקשת
- המשתמש יצא
- timeout
- service shutdown

דוגמה:

```
public async Task DoWorkAsync(CancellationToken ct)
{
```

```
while (!ct.IsCancellationRequested)
{
    await Task.Delay(1000, ct);
}
```

בריאיון חשוב להגיד:

לאורך השרשרת, במיוחד Cancellation Token אני משתדל להעביר במערכות production במערכות DB/HTTP/queue/background jobs.

מה בודקים — Multi-Service / Microservices עכשיו 15 שם

בדרך כלל מתכוונים, **multiservice** כשאומרים לך **מערכות**:

- services מערכת שמורכבת ממספר
- אחראי על תחום מסוים service כל
- מתקשרים ביניהם services

למשל:

- User Service
- Order Service
- Payment Service
- Notification Service

Multi-Service למה בכלל 16

יתרונות

- הפרדת אחריות
- service סקיילינג נפרד לכל
- deployment עצמאי
- קל יותר לחלק עבודה בין צוותים
- יכול להשתנות בלי להפיל הכול service כל

חסרונות

- services תקשורת בין
- monitoring מורכב יותר
- debugging קשה יותר
- consistency מורכב יותר
- latency
- failures בין services

משפט חזק לריאיון:

פותרים בעיות של סקייל וארגון, אבל יוצרים מורכבות תפעולית וארכיטקטונית Microservices.
microservices. לכן לא כל מערכת צריכה

מתקשרים Services איך (17)

יש שתי דרכים עיקריות:

סינכרוני

HTTP / REST / gRPC

ומחכה לתשובה Service B קורא ל־ Service A.

יתרונות:

- פשוט יחסית
- יש תשובה מיידיית

חסרונות:

- גבוה יותר coupling
- מושפע A, נופל B אם
- latency מצטבר

אסינכרוני

Message Broker:

- RabbitMQ
- Kafka
- Service Bus

Service A שולח event / message,
מטפל בזה אחר כך Service B.

יתרונות:

- loose coupling
- עמידות טובה יותר
- טוב scaling

חסרונות:

- יותר מורכב
 - eventual consistency
 - קשה יותר tracing
-

Multi-Service שאלות עומק על (18)

אחד נופל service מה קורה אם

תשובה טובה:

- retries מבוקרים
- circuit breaker
- timeout
- fallback
- queueing
- idempotency
- monitoring/alerts

idempotency מה זה

אם אותה בקשה נשלחת פעמיים — לא יוצר אפקט כפול.

דוגמה:

retry. אסור שחיוב יחויב פעמיים רק כי היה payments במערכת.

למה זה חשוב?

distributed במערכות:

- retries יש
- timeouts יש
- duplicate messages יש
- network failures יש

קריטי idempotency ולכן.

19) Consistency במערכות מרובות services

אחת DB אחת על transaction אפשר לעשות monolith במערכת.
זה יותר קשה multi-service.

למשל:

- Order Service יצר הזמנה
- Payment Service נכשל
- Notification Service כבר שלח הודעה

אז מה המצב?

פה נכנסים מושגים כמו:

- eventual consistency
- saga pattern
- compensating actions

אם ישאלו אותך לעומק, תענה כך:

services חוצה ACID transaction במערכות מבזרות קשה לשמור על
Saga, למשל, compensation, ובתהליכי eventual consistency לעיתים משתמשים ב-

20) פשוטה Saga מה זה

אחת גדולה transaction במקום,
יש רצף של צעדים.

לדוגמה:

1. יצירת הזמנה

2. payment intent שמירת
3. חיוב
4. עדכון מלאי
5. שליחת מייל

אם אחד השלבים נכשל

- מבצעים פיצוי
- למשל מבטלים הזמנה או מחזירים תשלום

Senior. רעיון חשוב מאוד ל-

21) Observability — Senior חשוב מאוד בריאיון

כבר לא מספיק לוג רגיל multi-service-

צריך:

- Logging
- Metrics
- Tracing
- Correlation ID

Correlation ID

מזהה אחד שעובר בין כל השירותים,
כדי שתוכל לעקוב אחרי בקשה מקצה לקצה.

למשל:

API Gateway נכנס ל- Request

Order Service עובר ל- →

- Payment Service
- Notification Service

אפשר למצוא הכל בלוגים, Correlation ID, אם יש.

תשובה יפה לריאיון:

Correlation ID, centralized logging, health checks, multi-service במערכת, production. אחרת קשה מאוד לחקור תקלות, distributed tracing, metrics.

מתי כן ומתי לא — 22) Retries

הרבה נופלים פה.

Retry אם טוב רק אם:

- הכשל זמני
- operation idempotent
- מתאים backoff יש

לא נכון אם Retry:

- אתה עלול ליצור חיוב כפול
- שכבר קורס service אתה מעמיס על
- timeout / circuit breaker אין

תשובה טובה:

idempotent ורצוי רק לפעולות, exponential backoff, צריכים להיות מוגבלים, עם retries או deduplication. עם מנגנון

23) מה זה Circuit Breaker

נופל service B אם,
לא צריך להמשיך להכות בו בלי סוף service A.

Circuit Breaker:

- מזהה כישלונות חוזרים
- "פותח מעגל"
- מפסיק זמנית קריאות
- נותן למערכת להתאושש

cascading failures זה מונע.

24) שאלות שיכולים לשאול אותך בריאיון

Senior: הנה שאלות אמיתיות בסגנון

על Task/Thread

- Process, Thread, Task? מה ההבדל בין
- thread יוצר async/await האם?
- Task.Run? מתי להשתמש ב־
- Thread Pool? מה זה
- race condition? מה זה
- deadlock? מה זה
- bottleneck איך תזהה?
- shared mutable state? הבעיה ב־

- lock contention? מה זה
- starvation? מה זה

על Multi-Service

- monolith ומתי microservices מתי תבחר
- מתקשרים services איך?
- אחד לא זמין service מה קורה אם?
- duplicate processing איך מונעים?
- observability איך מבטיחים?
- distributed transaction? מטפלים ב-
- eventual consistency? מה זה
- services במספר orders / payments / notifications איך תבנה מערכת
- scaling איך תבצע?
- message איך תתמודד עם?

ל-Monolith תשובה חזקה לשאלה: "מה ההבדל בין (25) Microservices?"

transactions, debugging, deployment, פשוט יותר לפיתוח Monolith.

עצמאי ועצמאות בין צוותים, אבל מוסיפים scaling, גותנים הפרדת אחריות Microservices. ותפעול consistency, מורכבות גבוהה של תקשורת, ניטור.

לכן הבחירה תלויה בגודל המערכת, הצוות, קצב השינויים והצרכים העסקיים.

זו תשובה מאוד טובה.

multi-service? תשובה חזקה לשאלה: "איך היית מתכנן מערכת (26)

אפשר לענות כך:

תהיה אחריות עסקית ברורה service ברורים, כדי שלכל bounded contexts קודם הייתי מגדיר חייבים להיות סינכרוניים ואיזה עדיף להעביר לאירועים flows לאחר מכן הייתי בוחר איזה אסינכרוניים.

idempotency, retries, timeouts, circuit breaker, centralized logging, tracing ו-health checks. הייתי מוסיף מבוקרים

happy path ולא רק failure scenarios בנוסף, הייתי מתכנן מראש

Team Lead / Architect. זה כבר נשמע

טעות נפוצה של מועמדים (27)

לא מספיק להגיד:

- "Thread זה יותר חדש מ־Task"
- "זה יותר טוב Microservice"
- "יותר מהיר async"

אלה תשובות חלשות.

במקום זה תגיד:

- O/I וזמן ל־CPU מה מתאים ל־
- concurrency מה הסיכונים של
- איך מונעים כפילויות
- failure איך מתמודדים עם
- איך מנטרים מערכת

גרסה קצרה שאתה יכול לזכור בעל פה (28)

אם אתה רוצה תשובת ריאיון קצרה:

הוא תוכנית רצה עם זיכרון מבודד Process.

Thread process הוא יחידת ביצוע בתוך Thread.

Thread שמייצג עבודה אסינכרונית או מקבילית, ולא בהכרח abstraction Task. NET. חדש.

parallelism אני שוקל CPU-bound threads, לא לחסום async/await אני מעדיף O/A. race conditions, deadlocks, locking, thread safety וcancellation. timeouts, retries, idempotency, circuit breaker, multi-service על חשבון במערכות כי הכשלון הוא חלק רגיל מהמערכת ולא מצב monitoring, tracing וeventual consistency. חריג.

"איך לענות אם שואלים אותך" תן דוגמה מהניסיון שלך (29)

אתה יכול לענות בסגנון כזה:

אני משתדל להפריד בין קריאות שחייבות תשובה מיידיית לבין services במערכות עם כמה תהליכי רקע.

ברור, ופעולות כמו timeout למשל, פעולות קריטיות למשתמש נעשות סינכרונית עם background או queue למערכות אחרות מועברות דרך sync או audit notifications, processing.

correlation id, retries לא מבוקר, ולהקפיד על shared state בנוסף אני משתדל להימנע מ-failures כדי שמערכת תישאר יציבה גם תחת idempotency מבוקרים ו-.

זה נשמע מצוין בריאיון גם אם לא נכנסים לטכנולוגיה מסוימת.

מה אני מציע לך עכשיו (30)

Senior. הנושא הזה מאוד חשוב לראיונות

:השלב הבא הכי טוב הוא לעשות לך אחד מאלה

1. Task / Thread / async / microservices דף תשובות מוכן לריאיון על
2. סימולציית ריאיון שבה אני שואל אותך שאלות ואתה עונה
3. שאלות ותשובות ברוסית על כל הנושא
4. concurrency / multi-service תרשימי זרימה פשוטים שמסבירים

Task, Thread, async ו-**microservices** אני ממליץ שנעשה עכשיו סימולציית ריאיון אמיתית בנושא

Thread פותח Task.Run מתי?

Senior ✎ שאלה מצוינת — זו בדיוק שאלה קלאסית של

:התשובה הקצרה

"Thread כמעט אף פעם לא פותח Task.Run"

.ברוב המקרים Thread Pool הוא משתמש ב-

◆ **Thread אז מתי כן נפתח?**

✕ **ברוב המקרים — לא**

כשאתה עושה

```
Task.Run (() => DoWork());
```

מה שקורה בפועל

- **Thread Pool** העבודה נכנסת ל-
- הקיימים מבצע אותה threads אחד ה-

כלומר:

Thread לא נוצר

◆ חדש Thread אז מתי כן ייווצר

1. צריך לגדול Thread Pool כשה-

זמינים (עומס גבוה) threads אם אין מספיק

- נוסף Thread יכול ליצור runtime ה-
- ("ידני" Thread לא) Thread Pool אבל זה עדיין חלק מה-

כלומר:

- חדש Thread כן נוצר
- עצמו `Task.Run` אבל לא בגלל
- Thread Pool אלא בגלל ניהול ה-

2. עם אופציה מיוחדת Task כששתמשים ב-

```
Task.Factory.StartNew(() => DoWork(),  
    TaskCreationOptions.LongRunning);
```

במקרה הזה:

☞ **Thread Pool לא מהיר** (ייעודי חדש Thread כן יוצר runtime)

למה?

כי אתה אומר:

"Thread Pool זו עבודה ארוכה, אל תבזבז"

3. Task לא דרך ידנית Thread כשאתה יוצר

```
new Thread(() => DoWork()).Start();
```

כאן:

☞ חדש Thread בוודאות נוצר

Task. אבל זה כבר לא קשור ל-.

◆ **Thread לא פותח Task.Run למה?**

כי:

- זה יקר Thread יצירת
- נותן Thread Pool:
- reuse
- scalability

- performance

◆ תשובה חזקה לריאיון

אם שואלים אותך

"חדש Thread פותח Task.Run מתי?"

תענה כך:

Thread Pool חדש באופן ישיר. הוא מתזמן את העבודה על Thread לא יוצר Task.Run מחליט להגדיל את עצמו תחת עומס, אבל זה Thread Pool חדש יכול להיווצר אם ה־ Thread לא בשליטה ישירה של Task.Run. או ליצור TaskCreationOptions.LongRunning ייעודי, ניתן להשתמש ב־ Thread אם רוצים ידנית Thread.

◆ שאלה ממשיכה שיכולים לשאול אותך

🔗 "Task.Run? למה לא להשתמש תמיד ב־?"

תשובה:

כדי לא לבזבז Task.Run, בלי async/await עדיף — I/O למשל, אם זו פעולת ASP.NET־ Thread.

או לקוד חוסם CPU-bound מתאים בעיקר ל־ Task.Run.

◆ סיכום קצר

Thread	חדש?	מצב
Task.Run	לא ✗	רגיל
Thread Pool	אולי ✓	עמוס
LongRunning	כן ✓	
new Thread()	כן ✓	

אם אתה רוצה — אני יכול לתת לך עכשיו:

🔥 (שמפילים הרבה מועמדים) Task/Thread שאלות טריקיות מהראיונות על

coupling מה זה

וגם בארכיטקטורה Senior שאלה מעולה — זה מושג **מאוד חשוב בראיונות**

◆ (קישוריות) Coupling מה זה

Coupling = רמת התלות בין רכיבים במערכת

במילים פשוטות:

כמה מחלקה / מודול / שירות אחד **תלוי** באחר

◆ הסבר הכי פשוט

אם יש לך שתי מחלקות

```
class OrderService
{
    private PaymentService _payment = new PaymentService();

    public void CreateOrder()
    {
        _payment.Pay();
    }
}
```

👉 כי, **coupling** גבוה כאן יש

- `OrderService` תלוי ישירות ב- `PaymentService`
- `OrderService` אתה עלול לשבור את `PaymentService` אם תשנה את

◆ Coupling סוגי

● Tight Coupling (קישוריות גבוהה — לא טוב)

כ"ש מחלקות "קשורות חזק"

- תלות ישירה
- קשה להחליף רכיבים
- קשה לבדוק (Unit Tests)
- שינוי קטן גורר שינויים בהרבה מקומות

דוגמה:

```
var payment = new PaymentService(); // תלות קשיחה
```

□ Loose Coupling (טוב — נמוכה)

כשיש פחות תלות:

- אפשר להחליף מימושים
- קל לבדוק
- מערכת יותר גמישה

דוגמה:

```
class OrderService
{
    private readonly IPaymentService _payment;

    public OrderService(IPaymentService payment)
    {
        _payment = payment;
    }
}
```

עכשיו ↩:

- אפשר להחליף מימוש
 - Mock אפשר לעשות
 - אין תלות ישירה
-

◆ משפט זהב לריאיון

זוהי רמת התלות בין רכיבים במערכת Coupling.

כדי לאפשר גמישות, תחזוקה קלה ויכולת שינוי בלי **loose coupling**, המטרה היא להגיע ל-
לשבור חלקים אחרים.

◆ Coupling במערכות Multi-Service

זוהי נהיה עוד יותר חשוב.

● Tight Coupling בין services

Service A → תשובה מיידית מ- Service B

בעיות:

- נופל A → נופל B אם
 - latency
 - תלות חזקה
-

□ Loose Coupling בין services

Service A → message שולח → Queue → Service B

יתרונות:

- לא תלוי בזמינות מיידית
- יותר יציב
- scalable

◆ דוגמה אמיתית (חשובה לראיון)

Tight coupling

- ישירות Payment Service קורא ל־ Order Service
- מחכה לתשובה

Loose coupling

- Order Service שולח event: "OrderCreated"
- Payment Service מאזין ומטפל

◆ Coupling מול Cohesion

שואלים את זה הרבה:

- **Coupling** (בין רכיבים) → רוצים נמוך
- **Cohesion** (בתוך רכיב) → רוצים גבוה

כלומר:

- (high cohesion) עושה דבר אחד טוב service כל
- (low coupling) לא תלויים חזק אחד בשני services

◆ בפועל Coupling איך מורידים

- Interfaces (`IPaymentService`)
 - Dependency Injection
 - Events / Message Queue
 - DTOs במקום מודלים פנימיים
 - API contracts יציבים
 - פנימי implementation לא לחשוף
-

◆ תשובה קצרה שאתה יכול לזכור

Coupling זה רמת התלות בין רכיבים.

Tight coupling אומר שתלות חזקה מקשה על שינוי ותחזוקה.

Loose coupling מאפשר גמישות, החלפה קלה של רכיבים וסקיילביליות — במיוחד במערכות multi-service.

◆ שאלת המשך קלאסית

☞ "במערכת קיימת coupling איך היית מפחית?"

Senior ☞ אם תרצה — אני אענה לך כמו בראיון

Task/Thread על Senior: מה ההבדל בין Process, Thread, Task? מה Thread Pool? מה זה Task.Run? מהי להשתמש ב-thread יוצר async/await האם זה הבעיה? במערכת מקבילית bottleneck איך תזהה? deadlock? מה זה race condition? Multi-Service על starvation? מה זה lock contention? מה זה shared mutable state? אחד לא service מה קורה אם מתקשרים services איך? monolith ומתי microservices תבחר איך מטפלים ב-observability איך מבטיחים? duplicate processing איך מונעים? זמין

distributed transaction? notifications / eventual consistency? תבנה מערכת מה זה / payments / orders במספר services? scaling? תבצע message איך תתמודד עם איך תבצע? פועמים?

אמיתי Senior / Team Lead מעולה — זה בדיוק סט שאלות של שתוכל כמעט לזכור בעל פה, אני אתן לך תשובות קצרות, מדויקות וברמה של ריאיון 🙌

◆ חלק 1 — Task / Thread / Concurrency

Process, Thread, Task? מה ההבדל בין ?

הוא תוכנית רצה עם זיכרון מבודד Process.

Thread process הוא יחידת ביצוע בתוך Thread.

Task עבודה אסינכרונית/מקבילית, ולא בהכרח משתמש ב-.NET abstraction הוא Thread חדש Thread.

thread יוצר async/await האם ?

לא.

O/וובזמן המתנה ל־ thread חדש — הוא מאפשר לא לחסום thread לא יוצר async/await runtime. וההמשך מתוזמן ע"י ה־

Task.Run? מתי להשתמש ב־ ?

הראשי, או כשעוטפים קוד חוסם thread שצריך להוריד מ־ CPU-bound כשיש עבודה אמיתי async עדיף — ASP.NET ב־ I/O לא להשתמש בו סתם לפעולות

? Thread Pool מה זה?

runtime. מנוהלים ע"י ה־ threads מאגר של
יותר יעיל וסקיילבילי — reuse חדש כל פעם, עושים thread במקום ליצור

? race condition מה זה?

במקביל והתוצאה תלויה בסדר הריצה state ניגשים לאותו threads מצב שבו כמה
מוביל לתוצאות לא צפויות.

? deadlock מה זה?

ואף אחד לא (locks) (או יותר) מחכים אחד לשני לנצח בגלל נעילות threads מצב שבו שני
מתקדם.

? במערכת מקבילית bottleneck איך תזהה?

באמצעות:

- profiling (CPU, memory)

- thread dumps
- metrics (latency, throughput)
- logging

ובדיקה אם יש:

- lock contention
 - thread starvation
 - blocking calls
-

? **shared mutable state? הבעיה ב-**

state משנים אותו threads כשכמה:

- race conditions נוצרים
- קשה לנבא התנהגות
- קשה לדבג

או סנכרון נכון immutable data עדיף.

? **lock contention? מה זה**

→ lock מנסים לקחת אותו threads מצב שבו הרבה
נוצרת המתנה → ירידה בביצועים.

? **starvation? מה זה**

משאבים כי אחרים תופסים הכל/CPU לא מקבל task או thread מצב שבו

◆ חלק 2 — Multi-Service / Architecture

monolith ומתי microservices מתי תבחר ?

Monolith:

- פשוט יותר
- קל לdebug/קל לפיתוח
- מתאים לצוות קטן

Microservices:

- כשיש סקייל גדול
 - צוותים מרובים
 - ברורים domains
- אבל מוסיף מורכבות גבוהה

מתקשרים services איך ?

דרכים 2:

- HTTP / gRPC: סינכרוני
 - Queue / Kafka / RabbitMQ: אסינכרוני
- ועמידות טובה יותר loose coupling אסינכרוני נותן.

אחד לא זמין service מה קורה אם ?

צריך:

- timeout
 - retry (מבוקר)
 - circuit breaker
 - fallback
- ולא להפיל את כל המערכת
-

duplicate processing? איך מונעים ?

באמצעות:

- idempotency keys
 - unique constraints
 - deduplication
 - message tracking
-

observability? איך מבטיחים ?

ע"י:

- logging מרכזי
 - metrics
 - distributed tracing
 - correlation ID
-

distributed transaction? איך מטפלים ב- ?

אחת גדולה transaction לא עושים

משתמשים ב:

- Saga pattern
 - eventual consistency
 - compensating actions
-

? eventual consistency מה זה ?

הנתונים לא מתעדכנים מיד בכל המערכת,
אבל עם הזמן מגיעים למצב עקבי.

? notifications / payments / orders איך תבנה מערכת ?

דוגמה טובה:

- Order Service → יוצר הזמנה
- event: OrderCreated שולח
- Payment Service חיוב ומבצע
- Notification Service שולח מייל

🔗 הפרדה + async → loose coupling

? scaling איך תבצע ?

כמה רמות:

- scale out (עוד instances)
 - load balancer
 - cache (Redis)
 - queue ל־background work
 - DB scaling (read replicas)
-

שנצרך פעמיים message איך תתמודד עם ?

distributed. זה קורה תמיד במערכות

פתרונות:

- idempotent handlers
 - message ID tracking
 - deduplication table
 - בדיוק-פעם (at-least-once + idempotency)
-

תשובת "וואו" לריאיון (אפשר להגיד בסוף) 🔥

שימוש נכון ב־ shared mutable state, במערכות מקבילות אני מתמקד בהימנעות מ־ lock contention כמו bottlenecks ובזיהוי CPU, ל־parallelism ו־O/אל־async/await starvation.

multi-service מראש מתכנן אני failure scenarios — retries, timeouts, circuit breakers — idempotency — loose כדי להשיג event-driven architecture ומשתמש ב־ וסקיילביליות coupling.

אם אתה רוצה לעלות רמה עוד יותר 🧠

אני יכול לעשות לך עכשיו אחד מאלה:

1. אני שואל, אתה עונה — **אמיתי Mock Interview** 🗣️
2. **Seniors שאלות טריקות שמכשילות** ❌
3. כמו בגוגל/אמזון **System Design (Orders + Payments) תרגיל** 📄
4. להסביר הכל ברוסית כמו שאתה אוהב לפעמים ru

🔗 תגיד מה אתה רוצה — ונלך עמוק

Seniors שאלות טריקות שמכשילות

Seniors מעולה 😊 — עכשיו נכנסים לרמה של **שאלות שממש מפילות גם**

.אני אתן לך כל שאלה + מה רוב האנשים עונים (לא נכון) + תשובה נכונה + למה זה טריק

🔥 1 חלק — Concurrency / Task / Thread

? **עושה קוד מקבילי** `async/await` **האם 1.**

✗ תשובה נפוצה (לא נכונה)

כן, זה מריץ דברים במקביל

✓ **תשובה נכונה**

לא מבטיח מקביליות `async/await`. לא.

בזמן המתנה thread הוא מאפשר **לא לחסום**.

במקביל Tasks מקביליות מתקבלת רק אם מפעילים כמה.

טריק

אנשים מבלבלים בין:

- `async` = לא חוסם
- `parallel` = כמה דברים רצים יחד

מה יקרה כאן? 2.

```
await Do1();  
await Do2();
```

תשובה נפוצה X

זה רץ במקביל

תשובה נכונה ✓

הסתיים Do1 מתחיל רק אחרי ש־ Do2 — זה רץ סדרתי

איך כן מקבילי? ✓

```
var t1 = Do1();  
var t2 = Do2();  
await Task.WhenAll(t1, t2);
```

ASP.NET? תמיד רע ב־ `Task.Run` האם? 3.

תשובה נפוצה X

כן, לא להשתמש אף פעם

✓ תשובה נכונה

לא תמיד רע.

I/O כשמשתמשים בו ל-

או קוד חוסם CPU-bound שימושי ל-

🎯 טריק

מחפשים חשיבה, לא דוגמה עיוורת.

? 4. כמה הבעיה בקוד הזה?

```
public async Task<IActionResult> Get()
{
    var result = SomeAsync().Result;
    return Ok(result);
}
```

✗ תשובה נפוצה

זה עובד

✓ תשובה נכונה

deadlock עלול לגרום ל-

async על thread כי אתה חוסם

✓ □ נכון:

```
var result = await SomeAsync();
```

? 5. race condition תמיד פותר lock האם?

✗ תשובה נפוצה

כן

✓ תשובה נכונה

לא תמיד:

- deadlock יכול לגרום ל-
 - contention יוצר
 - distributed systems לא מתאים ל-
-

? למחזה מסוכן. 6.

```
Parallel.ForEach(list, item =>
{
    Console.WriteLine(item);
});
```

✗ תשובה נפוצה

אין בעיה

✓ תשובה נכונה

Console זה shared resource → יכול לגרום:

- contention
 - output מבולגן
 - performance issues
-

? פותר הכל ConcurrentDictionary האם 7.

✗ תשובה נפוצה

thread-safe כן, זה

✓ תשובה נכונה

בודדות בטוחות operations רק
מורכב atomic flow לא מבטיח

? 8. מה הבעיה כאן?

```
if (!dict.ContainsKey(key))  
{  
    dict[key] = value;  
}
```

✗ תשובה נפוצה

תקין

✓ תשובה נכונה

race condition
יכולים להיכנס יחד threads שני

נכון: ☐ ✓

```
dict.TryAdd(key, value);
```

? 9. חוסך תמיד משאבים async האם?

✗ תשובה נפוצה

כן

✓ תשובה נכונה

לא:

- overhead של state machine
- context switching
- CPU-bound לא מתאים ל-

? ואיפה רואים את זה Thread Starvation מה זה 10.

✗ תשובה נפוצה

לא בטוח

✓ תשובה נכונה

נגמר Thread Pool כש-
לרוב קורה כשעושים:

```
.Result / .Wait()
```

🔥 חלק 2 — Multi-Service / System Design

? תמיד עדיפים microservices האם 11.

✗ תשובה נפוצה

scalable כן, זה

✓ תשובה נכונה

לא — מוסיף:

- latency
 - complexity
 - קשה debugging
-

? 12. retry? הבעיה ב-

✗ תשובה נפוצה

אין, זה טוב

✓ תשובה נכונה

יכול לגרום:

- duplicate actions
 - overload
 - cascading failure
-

? 13. (exactly once) איך מבטיחים בדיוק פעם אחת?

✗ תשובה נפוצה

עושה את זה queue

✓ תשובה נכונה

exactly once אין באמת
משתמשים:

- at-least-once + idempotency

14. event-driven? הבעיה ב-?

✗ תשובה נפוצה

אין

✓ תשובה נכונה

- קשה debugging
- קשה tracing
- eventual consistency
- בעייתי ordering

15. מה הבעיה כאן?

Order Service → Payment → Inventory → Notification

✗ תשובה נפוצה

תקין

✓ תשובה נכונה

chain סינכרוני → fragile:

- אם אחד נופל → הכל נופל
- גבוה latency

16. double payment? איך תמנע?

✗ תשובה נפוצה

retry

✓ תשובה נכונה

idempotency key + DB constraint

? 17. distributed transaction למה?

✗ תשובה נפוצה

לא בעיה

✓ תשובה נכונה

- slow
 - לא scalable
 - גבוה coupling
→ Saga עדיף
-

? 18. shared DB בין services? הבעיה ב-

✗ תשובה נפוצה

יותר פשוט

✓ תשובה נכונה

- tight coupling
- breaking changes
- אין isolation

? יגיע פעמיים message מה יקרה אם 19?

✗ תשובה נפוצה

לא יקרה

✓ תשובה נכונה

קורה תמיד

→ idempotent handler צריך

? 20. backpressure? מה זה

✗ תשובה נפוצה

לא יודעים

✓ תשובה נכונה

כשמערכת לא עומדת בעומס

וצריך:

- throttle
 - queue
 - reject requests
-

🔥 שאלת "וואו" (מפילה הרבה)

? יכול לשבור ביצועים async למה

תשובה נכונה:

כי:

- state machine יוצר
- allocation
- context switching
- CPU-bound לא מתאים ל-
- יותר מהיר sync לפעמים

🔥 משפט סיום שגורם למראיין להתרשם

אלא עם הבנה של tools, לא נפתרות רק עם distributed systems ו-concurrency הרבה בעיות בין ביצועים, עקביות, זמינות ופשטות — trade-offs.

🚀? רוצה לעלות עוד רמה

אני יכול לתת לך:

1. 🎯 **Mock Interview** בגוגל
2. 📄 **System Design** אמיתי (Payments / Orders)
3. 🧠 **שאלות "מלכודת" עוד יותר קשות** (Architect level)
4. 📖 הכל ברוסית עם דוגמאות עמוקות ru

👉 תגיד מה בא לך — ונעלה אותך רמה